



Propuesta de Trabajo Profesional de Ingeniería en Informática

Oxidized Neural Orchestra

Sistema distribuido de entrenamiento de modelos de aprendizaje profundo en Rust para implementar y comparar estrategias de distribución como Parameter Server y All-Reduce.

Tutor: Ing. Ricardo Andrés Veiga
rveiga@fi.uba.ar

Co-Tutor: Dr. Ing. José Ignacio Alvarez Hamelin
ihameli@fi.uba.ar

Alumnos

Alejo Ordoñez (*Padrón 108397*)
alordonez@fi.uba.ar

Lorenzo Minervino (*Padrón 107863*)
lminervino@fi.uba.ar

Marcos Bianchi Fernández (*Padrón 108921*)
mbianchif@fi.uba.ar

Facultad de Ingeniería, Universidad de Buenos Aires

Índice

1	Acta de Acuerdo y aval del director	3
2	Lugar para realizar el proyecto	4
3	Resumen	5
4	Palabras clave	6
5	Abstract	7
6	Keywords	8
7	Introducción	9
8	Estado del arte	10
9	Problema detectado	11
10	Solución propuesta	12
10.1	Tecnologías	12
11	Evaluación preliminar de impacto económico, social y ambiental	14
12	Metodología	15
12.1	Gestión y roles	15
12.2	Proceso de desarrollo	15
12.3	Criterios de aceptación	15
12.4	Artefactos de gestión e indicadores	16
12.5	Riesgos iniciales	16
13	Experimentación, validación y control de calidad	18
14	Plan de actividades	19
14.1	Entregables e hitos de avance	20
14.2	Carga horaria	20
15	Aportes individuales	22
16	Referencias	23
17	Anexos	24
17.1	Anexo A: Repositorio del proyecto	24

1 Acta de Acuerdo y aval del director

En la Ciudad Autónoma de Buenos Aires, al día veinticinco del mes de noviembre del año dos mil veinticinco, se reúnen el profesor Ing. Ricardo A. Veiga y el profesor Dr. Ing. J. Ignacio Alvarez-Hamelin con los estudiantes de la carrera de Ingeniería en Informática el Sr. Alejo Ordoñez (Padrón 108397, DNI 44480134), el Sr. Lorenzo Minervino (Padrón 107863, DNI 42720539) y el Sr. Marcos Bianchi Fernández (Padrón 108921, DNI 43874791). El objetivo de la reunión es acordar el tema de Trabajo Profesional de Ingeniería en Informática.

Luego de haber conversado sobre las áreas de interés de los estudiantes, se acuerda establecer como Tema de Trabajo Profesional en Ingeniería en Informática: “Oxidized Neural Orchestra”.

El profesor Ing. Ricardo A. Veiga acepta dirigir el Trabajo Profesional “Oxidized Neural Orchestra”, que van a desarrollar los estudiantes de la carrera de Ingeniería en Informática el Sr. Alejo Ordoñez (Padrón 108397, DNI 44480134), el Sr. Lorenzo Minervino (Padrón 107863, DNI 42720539) y el Sr. Marcos Bianchi Fernández (Padrón 108921, DNI 43874791). El profesor Dr. Ing. J. Ignacio Alvarez-Hamelin acepta la función de co-tutor para dicho trabajo.

Tanto el Director como los estudiantes declaran conocer la Resolución de Consejo Directivo 1124 de 2025, que define la normativa para los Trabajos Integradores Finales para carreras de grado a partir de 2026, y las obligaciones de ambas partes establecidas en la misma. En particular, para el cumplimiento del necesario contacto periódico, se comprometen a reunirse con una periodicidad igual o menor a las dos semanas, sea de manera presencial o remota, y acuerdan respetar la forma de trabajo definida en esta propuesta.

Firma y aclaración del director:

Ing. Ricardo A. Veiga _____

Dr. Ing. José Ignacio Alvarez Hamelin _____

Firmas y aclaraciones de los estudiantes:

Alejo Ordoñez _____

Lorenzo Minervino _____

Marcos Bianchi Fernández _____

2 Lugar para realizar el proyecto

A definir.

3 Resumen

El entrenamiento de modelos de aprendizaje profundo, requiere de grandes recursos computacionales y largos tiempos de ejecución; los algoritmos que se utilizan hacen uso intensivo de CPU y requieren de capacidades de memoria muy grandes para poder almacenar tanto los datos de entrenamiento, como el modelo en sí. Junto con la revolución de la aplicación de la inteligencia artificial, se ha vuelto un tema candente el minimizar éste tiempo de entrenamiento. Para lograrlo, la estrategia más escalable es su ejecución distribuida.

El problema es que utilizar un algoritmo distribuido de optimización para entrenar un modelo de aprendizaje profundo viene acompañado de desafíos a la hora de plantear la estrategia de distribución, y de peores (o directamente inutilizables) resultados de convergencia si no se realiza con cuidado. Por ejemplo, si se distribuyera por datos y cada nodo optimizara por separado sobre la partición que le tocó para luego combinar los resultados calculados de forma independiente, el modelo no quedaría optimizado sobre la totalidad del dataset, dado que cada nodo habría incurrido en mucho *overfitting* sobre su partición. Además, esas soluciones especializadas no son directamente combinables mediante un promedio de sus pesos, ya que el resultado de cada iteración de los algoritmos de optimización depende del anterior.

A pesar de las dificultades que existen, entre ellas las ya mencionadas, hay varias soluciones distribuidas que logran sortearlas y que proveen grandes reducciones en los tiempos de ejecución de los algoritmos de entrenamiento, a partir de la suma del poder de cómputo y de la capacidad de memoria de los nodos en los sistemas que las ejecutan. En la actualidad, dos de las soluciones más difundidas son *Parameter Server* y *All-Reduce*. *All-Reduce* es sincrónico: en cada paso todos los nodos aplican la misma actualización, lo que favorece la coherencia de la optimización. *Parameter Server*, en cambio, centraliza los parámetros y admite variantes tanto sincrónicas como asincrónicas; en su modo asincrónico prioriza la reducción del tiempo de entrenamiento, a costa de esa coherencia. Comparar ambos enfoques, teniendo en cuenta las variantes de *Parameter Server*, es uno de los ejes de este trabajo.

En este trabajo se estudian estos algoritmos y sus derivados, y se implementa un sistema distribuido de entrenamiento que sirva como base para su análisis y comparación, y como punto de partida para el desarrollo de mejoras.

4 Palabras clave

Aprendizaje profundo, entrenamiento distribuido, sistemas distribuidos, paralelismo de datos, *Parameter Server*, *All-Reduce*, *Strategy-Switch*, convergencia, escalabilidad, Rust.

5 Abstract

Training deep learning models requires large computational resources and long execution times; the algorithms involved make intensive use of the CPU and demand very large memory capacities in order to store both the training data and the model itself. Together with the revolution in the application of artificial intelligence, minimizing this training time has become a pressing topic. To achieve this, the most scalable strategy is distributed execution.

The problem is that using a distributed optimization algorithm to train a deep learning model comes with challenges in designing the distribution strategy, and with worse (or directly unusable) convergence results if it is not done carefully. For instance, if the workload were distributed by data and each node optimized separately over the partition assigned to it, later combining the independently computed results, the model would not be optimized over the whole dataset, since each node would have incurred significant *overfitting* on its own partition. Moreover, those specialized solutions are not directly combinable by averaging their weights, because the result of each iteration of the optimization algorithms depends on the previous one.

Despite the existing difficulties, among them the ones already mentioned, there are several distributed solutions that manage to overcome them and provide large reductions in the execution times of training algorithms, by adding up the computing power and memory capacity of the nodes in the systems that run them. Nowadays, two of the most widespread solutions are *Parameter Server* and *All-Reduce*. *All-Reduce* is synchronous: at every step all nodes apply the same update, which favours the coherence of the optimization. *Parameter Server*, in contrast, centralizes the parameters and admits both synchronous and asynchronous variants; in its asynchronous mode it prioritizes the reduction of training time, at the expense of that coherence. Comparing both approaches, taking into account the variants of *Parameter Server*, is one of the axes of this work.

In this work these algorithms and their derivatives are studied, and a distributed training system is implemented to serve as a basis for their analysis and comparison, and as a starting point for the development of improvements.

6 Keywords

Deep learning, distributed training, distributed systems, data parallelism, *Parameter Server*, *All-Reduce*, *Strategy-Switch*, convergence, scalability, Rust.

7 Introducción

El entrenamiento de modelos de aprendizaje profundo requiere de grandes recursos computacionales y largos tiempos de ejecución. Junto con la revolución de la aplicación de la inteligencia artificial, minimizar ese tiempo de entrenamiento se ha vuelto un tema candente, y la estrategia más escalable para lograrlo es su ejecución distribuida. Sin embargo, distribuir el entrenamiento no es directo: introduce desafíos en el diseño de la estrategia de distribución y puede degradar la convergencia de la optimización si no se realiza con cuidado.

Este trabajo se propone estudiar en profundidad las principales estrategias de entrenamiento distribuido y, a partir de dicho análisis, implementar un sistema distribuido de entrenamiento de modelos de aprendizaje profundo que sirva como base para la comparación de esas estrategias y como punto de partida para el desarrollo de posibles mejoras.

El presente documento se organiza de la siguiente manera. En el *Estado del arte* se realiza un relevamiento de los avances relacionados al entrenamiento distribuido y se presentan los dos enfoques fundamentales que sirven como base al trabajo. En *Problema detectado* se precisa la dificultad central que se aborda. En *Solución propuesta* se describe, a grandes rasgos, el sistema a construir y las tecnologías con las que se lo desarrollará. Luego se presenta una *Evaluación preliminar de impacto*, la *Metodología* de trabajo, la estrategia de *Experimentación, validación y control de calidad*, el *Plan de actividades* y los *Aportes individuales*. Finalmente se incluyen las *Referencias* y los *Anexos*.

8 Estado del arte

El entrenamiento distribuido surge como una extensión de las técnicas de paralelización en *HPC* (High Performance Computing). Con los primeros enfoques de paralelismo de datos en frameworks como **TensorFlow** (Developers, 2021) y **PyTorch** (Paszke et al., 2019), y sus implementaciones más recientes optimizadas para arquitecturas heterogéneas, la evolución de estos métodos refleja la tensión constante entre escalabilidad y coherencia de los parámetros del modelo.

El equilibrio entre convergencia y eficiencia de comunicación ha motivado una amplia línea de investigación. En la actualidad, existen dos enfoques fundamentales que sirven como base para el desarrollo de nuevas técnicas: **Parameter Server** (M. Li et al., 2014) y **All-Reduce** (Z. Li, Davis, & Jarvis, 2017).

En el enfoque de Parameter Server, el modelo se divide en un conjunto de parámetros centralizados (no necesariamente en un único nodo), a los cuales los nodos trabajadores envían sus gradientes y desde los cuales reciben las actualizaciones, que surgen de la agregación de estas contribuciones. Este esquema admite tanto una coordinación sincrónica, en la que el servidor espera los gradientes de todos los trabajadores antes de actualizar el modelo, como una asincrónica, en la que aplica cada gradiente apenas llega; esta última mejora la escalabilidad y reduce el tiempo de entrenamiento, aunque a costa de la coherencia entre copias del modelo.

En contraste, en All-Reduce todos los nodos intercambian gradientes de manera directa (utilizando, por ejemplo, implementaciones en anillo) y avanzan de forma sincrónica: en cada paso todos aplican la misma actualización, lo que preserva la coherencia entre réplicas. El precio es el costo de comunicación, que crece con el número de nodos, aunque topologías como el anillo lo atenúan.

Uno de los claros casos de combinación de estos algoritmos es **Strategy-Switch** (Provatas, Chalas, Konstantinou, & Koziris, 2025), que inicia el entrenamiento de forma sincrónica sobre All-Reduce y, guiado por una regla empírica, continúa con Parameter Server asincrónico una vez que el modelo en entrenamiento se estabiliza; logrando así combinar la precisión del régimen sincrónico de *All-Reduce* con la reducción de tiempo del régimen asincrónico de *Parameter Server*.

En este trabajo se estudian en profundidad ambos algoritmos base y se investigan los métodos que de ellos derivan, con el objetivo de proponer mejoras mediante la combinación de estrategias o la optimización de sus componentes de comunicación y sincronización.

9 Problema detectado

El principal desafío del desarrollo de algoritmos distribuidos de entrenamiento de modelos de aprendizaje profundo es encontrar una estrategia que disminuya satisfactoriamente el tiempo del entrenamiento, o que provea una solución a la incapacidad de un flujo existente de almacenar el dataset o incluso el modelo en sí. Esto sería mucho más sencillo si las iteraciones no fueran dependientes; podríamos simplemente utilizar una estrategia análoga a un *fork-join* y combinar la solución. Pero en un *gradient-descent*, la dirección de un descenso en la superficie de costo depende del punto en el que se encuentra el proceso en un momento dado, es decir, depende de la configuración de la matriz de pesos del modelo en esa iteración.

En un esquema de paralelización del entrenamiento por datos, existe el riesgo de incurrir en *overfitting* sobre las particiones si los pesos no se sincronizan con la frecuencia suficiente. Por otro lado, si la sincronización se realiza con demasiada frecuencia, el tiempo de comunicación entre los nodos puede volverse dominante. Este costo no es en absoluto despreciable: si la comunicación representa una fracción significativa del tiempo total de entrenamiento, la distribución del cómputo pierde sentido, ya que la ejecución paralela termina siendo más lenta que el entrenamiento secuencial.

Adicionalmente, no alcanza con dejar que cada nodo optimice por separado hasta una solución especializada sobre su partición y luego combinar esos modelos mediante un promedio de sus pesos, ya que el resultado de cada iteración de los algoritmos de optimización que se utilizan para el entrenamiento depende del anterior y esas soluciones especializadas no son directamente combinables. Por eso las estrategias distribuidas sincronizan durante el entrenamiento, agregando los gradientes de los nodos en cada paso en lugar de promediar modelos ya entrenados por separado. Este equilibrio entre convergencia y eficiencia de comunicación es, en definitiva, el problema que este trabajo aborda al estudiar y comparar las distintas estrategias de distribución.

10 Solución propuesta

Se propone implementar un sistema distribuido de entrenamiento de modelos de aprendizaje profundo que sirva como base para la investigación de los trabajos previos, actuales y que surjan sobre esta temática. La idea es que sea un sistema *parametrizable*, de modo de facilitar la posterior comparación entre estrategias de distribución y el desarrollo de mejoras que optimicen los tiempos de ejecución.

Sobre esa base se implementarán los algoritmos que hoy sirven como punto de partida del área: *Parameter Server*, *All-Reduce* y *Strategy-Switch*. Estas implementaciones funcionarán como referencia para la comparación con las futuras mejoras que se estudien y desarrollen, tanto en la comunicación entre nodos como en la sincronización de las copias del modelo. En términos generales, el desarrollo del trabajo conlleva:

1. Desarrollar un sistema distribuido de entrenamiento de modelos de aprendizaje profundo en Rust.
2. Implementar y comparar los distintos algoritmos que se utilicen para la ejecución del entrenamiento distribuido, sobre el sistema implementado.
3. Proveer una biblioteca en Python, construida sobre una interfaz de funciones externas (FFI), que permita configurar y utilizar el sistema desarrollado desde ese lenguaje.
4. Desarrollar una interfaz interactiva de terminal (TUI) que permita configurar, lanzar y monitorear en vivo los entrenamientos distribuidos sobre el sistema.
5. Simular la ejecución sobre distintas configuraciones del sistema distribuido, para así obtener datos que se puedan analizar, y obtener comparaciones de los distintos algoritmos que se implementen, usando Python.
6. Confeccionar un informe detallado de la evolución del trabajo y los resultados obtenidos.

10.1 Tecnologías

Las tecnologías que van a ser utilizadas para el desarrollo de este proyecto son:

- **Rust:** Se opta por el lenguaje de programación Rust para la implementación del sistema principal, porque ofrece: en primer lugar, la capacidad de editar código a bajo nivel, por la robustez del lenguaje, siendo que los requerimientos mínimos de compilación son más estrictos que la mayoría del resto de lenguajes, y que ofrece *fearless-concurrency*, haciendo chequeos estáticos de posibles problemas con la concurrencia de los programas, y por la relevancia que está cobrando en estos últimos tiempos.
- **Python:** Se va a usar Python para el análisis de datos obtenidos a partir de las comparaciones de los distintos algoritmos de machine-learning distribuido que serán llevados a cabo en el desarrollo del trabajo, y, por ser uno de los lenguajes más utilizados en la industria del aprendizaje profundo, para implementar una Interfaz de Funciones Externas (en inglés Foreign Function Interface, FFI) del resultado del sistema principal.
- **Docker:** Se hará uso de Docker para simular la ejecución del sistema en distintos entornos, y para agilizar la automatización de esto último.

Estas tres tecnologías constituyen la base del trabajo y están definidas desde el inicio. Las restantes decisiones tecnológicas se tomarán a medida que avance el desarrollo, en particular las bibliotecas concretas para la serialización de tensores y para el transporte de mensajes entre nodos, la biblioteca de vinculación entre Rust y Python sobre la que se construya la FFI, y la eventual herramienta de

integración continua. Para definir las se tendrán en cuenta tres consideraciones: que no introduzcan un costo de comunicación o de serialización que distorsione la comparación entre las estrategias de distribución, que se integren con el sistema de construcción y de pruebas del ecosistema de Rust sin requerir dependencias externas al mismo, y que su licencia sea de código abierto, de modo que el sistema resultante pueda ser retomado y extendido por terceros.

11 Evaluación preliminar de impacto económico, social y ambiental

La siguiente evaluación es de carácter preliminar y descriptivo, realizada con los conocimientos disponibles al momento de la propuesta. Se organiza en torno a tres ejes, económico, social y ambiental, sobre los que el desarrollo de un sistema de entrenamiento distribuido puede tener incidencia.

Impacto económico. El entrenamiento de modelos de aprendizaje profundo concentra buena parte de su costo en el tiempo de cómputo y en la infraestructura necesaria para sostenerlo. Al distribuir el entrenamiento sobre varios nodos, el sistema propuesto apunta a reducir ese tiempo y, con él, el costo asociado, sumando el poder de cómputo y la capacidad de memoria de máquinas ya existentes. En particular, la atención puesta en configuraciones heterogéneas permite aprovechar hardware disímil y no necesariamente de última generación, lo que reduce la barrera de entrada frente a la alternativa de adquirir equipamiento especializado.

Impacto social. Al tratarse de una base abierta y parametrizable, el trabajo busca acercar el entrenamiento distribuido a equipos que no cuentan con grandes clusters, y ofrecer un punto de partida para la investigación y la docencia en el área. La disponibilidad de implementaciones de referencia de los algoritmos fundamentales, junto con una interfaz accesible desde Python, facilita que otros retomen, comparen y extiendan el trabajo.

Impacto ambiental. El tiempo de cómputo se traduce de forma directa en consumo energético. En este sentido, cualquier reducción del tiempo total de entrenamiento tiene un correlato en el consumo, aunque la distribución introduce un costo adicional de comunicación entre nodos que debe ser tenido en cuenta. Justamente, el estudio de optimizaciones en la comunicación y la sincronización, centrales en este trabajo, apunta a que la ejecución distribuida traduzca su ganancia de tiempo en un uso más eficiente de los recursos, y no solo en un mayor consumo agregado.

12 Metodología

12.1 Gestión y roles

Se establece un compromiso por parte de cada estudiante para dedicar un total de 500 horas al desarrollo del trabajo profesional. Esto representa, en promedio, unas 16 horas semanales por persona a la ejecución de las tareas asignadas. Este compromiso se mantendrá a lo largo de 32 semanas (dos cuatrimestres). Además, se tiene previsto llevar a cabo encuentros periódicos en formato virtual entre los miembros del equipo y los tutores, cada semana. El propósito de estas reuniones es informar el avance y desarrollo del proyecto en curso. Asimismo, se abordarán aspectos como la definición de prioridades en las labores a realizar y la planificación requerida para la próxima etapa del proceso.

En cuanto a los roles, el Ing. Ricardo A. Veiga cumple la función de tutor y el Dr. Ing. J. Ignacio Alvarez-Hamelin la de co-tutor, orientando el desarrollo y revisando periódicamente el avance. Los tres estudiantes realizan todas las etapas de análisis, implementación y pruebas del proyecto.

12.2 Proceso de desarrollo

El método de desarrollo es incremental: el trabajo se organiza en iteraciones con entregables intermedios y una cadencia de prácticas de seguimiento y mejora continua. Las tareas de desarrollo, prueba y despliegue que se puedan automatizar estarán automatizadas, y todo el código y los artefactos que requieran versionarse se gestionan con una herramienta de control de versiones.

El código y los artefactos que requieren versionarse se gestionan con **Git**, sobre un repositorio alojado en **GitHub**. El trabajo se organiza en ramas por funcionalidad que se integran mediante *pull requests*, y los mensajes de los *commits* siguen la convención de *Conventional Commits*, lo que mantiene un historial legible y trazable de la evolución del proyecto.

El seguimiento del proceso, la gestión de tickets y el registro de errores se realizan con las *issues* del repositorio en GitHub, categorizadas mediante etiquetas por tipo (*bug*, *enhancement*, *documentation*) y por componente del sistema (por ejemplo, *parameter-server*, *comms*, *worker*, *ffi*, *tui*).

En cuanto a la automatización, la construcción y las pruebas del sistema se ejecutan con las herramientas del ecosistema de Rust, y los entornos de simulación se levantan de forma automatizada mediante scripts sobre **Docker**, lo que hace reproducible el despliegue de las distintas configuraciones de nodos.

Algunas cuestiones metodológicas todavía no están definidas a esta altura, como la incorporación de un flujo de integración continua que ejecute automáticamente las pruebas ante cada cambio; su adopción dependerá de la evolución del proyecto y de las necesidades que surjan durante el desarrollo.

12.3 Criterios de aceptación

Cada *pull request* se somete a revisión de código por parte de otro integrante antes de integrarse. Una entrega se considera aceptada cuando se cumplen, de forma conjunta, tres condiciones: la revisión de código fue aprobada, la totalidad de las pruebas automatizadas asociadas se ejecuta con éxito, y la funcionalidad requerida queda demostrada mediante una prueba de aceptación. Para las funcionalidades que involucran la coordinación entre nodos, esa prueba de aceptación consiste en una

ejecución completa de entrenamiento distribuido sobre el entorno simulado en Docker, verificando que la convergencia obtenida sea consistente con la del entrenamiento secuencial equivalente.

12.4 Artefactos de gestión e indicadores

Como artefactos de gestión se llevan minutas de las reuniones periódicas, el registro del alcance y el registro de riesgos del proyecto, este último revisado en cada reunión de seguimiento. Los errores registrados como *issues* se clasifican además por criticidad, distinguiendo aquellos que impiden el avance del trabajo de los que admiten una corrección diferida, lo que permite priorizar su tratamiento dentro de cada iteración.

Sobre la calidad del proceso se definen y miden los siguientes indicadores: el tiempo transcurrido entre la apertura y el cierre de una *issue*, la proporción de *pull requests* que requieren correcciones tras la revisión de código, y el desvío entre las horas planificadas en el Plan de actividades y las horas efectivamente dedicadas a cada tarea. Este último funciona a la vez como indicador de tiempos; dado que el trabajo se desarrolla sobre hardware propio y con herramientas de código abierto, el costo del proyecto se reduce a las horas-persona invertidas.

Sobre la calidad del producto se consideran la cobertura de las pruebas automatizadas, la proporción de pruebas exitosas sobre el total, y la cantidad de advertencias reportadas por las herramientas de análisis estático del lenguaje. Estos indicadores se relevan de forma periódica y se discuten en las reuniones de seguimiento, de modo que su evolución sirva para reajustar las prioridades del trabajo.

12.5 Riesgos iniciales

Se identifican los siguientes riesgos iniciales, junto con las medidas previstas para mitigarlos:

- **Complejidad de la implementación distribuida en Rust.** La coordinación entre nodos y el manejo de la concurrencia son intrínsecamente complejos y propensos a errores sutiles, como bloqueos o condiciones de carrera. Como mitigación, el desarrollo es incremental y se apoya en las garantías de *fearless-concurrency* del lenguaje y en una cobertura de pruebas unitarias y de integración que acompaña cada avance.
- **Disponibilidad de hardware para simulaciones representativas.** Reproducir un entorno distribuido realista requiere de múltiples máquinas, algo no siempre disponible. Para mitigarlo, las simulaciones se ejecutan sobre contenedores Docker con límites de recursos por nodo, lo que permite emular configuraciones heterogéneas de forma controlada; se asume que los resultados así obtenidos son una aproximación y no un reemplazo exacto de un despliegue sobre máquinas físicas separadas.
- **Dependencia de trabajos previos.** Parte del trabajo se apoya en algoritmos y resultados publicados, como *Strategy-Switch*, cuya interpretación e implementación pueden diferir de lo documentado. La mitigación consiste en implementar versiones de referencia de los algoritmos base y validarlas antes de construir mejoras sobre ellas.
- **Amplitud del alcance y estimación de tiempos.** El trabajo abarca tanto el sistema base como varias líneas de optimización (comunicación, sincronización y carga en configuraciones heterogéneas), lo que dificulta la estimación del esfuerzo. Para mitigarlo, se prioriza primero un sistema base funcional junto con las implementaciones de referencia, y las optimizaciones se

abordan de forma incremental según el avance, con reuniones semanales de seguimiento para reajustar prioridades.

13 Experimentación, validación y control de calidad

La validación del trabajo se apoya en dos frentes complementarios. Por un lado, el sistema desarrollado se prueba con tests unitarios y de integración, que verifican el comportamiento de sus componentes de forma aislada y en conjunto. Por otro lado, se simula la ejecución de los algoritmos implementados sobre el sistema distribuido base en distintas configuraciones de nodos, con el fin de obtener métricas que muestren el rendimiento del sistema según los parámetros que este use, para distintas combinaciones de máquinas con distintas capacidades de cómputo. Los resultados obtenidos de la comparación de los algoritmos se analizan y se vuelcan utilizando gráficos en Python. El conjunto de pruebas se entrega como resultado final del proyecto y como verificación del mismo.

Pruebas y automatización. El sistema se verifica con tres tipos de pruebas. Las pruebas unitarias ejercitan cada componente de forma aislada; las pruebas de integración validan la interacción entre nodos durante una ejecución distribuida; y las pruebas de aceptación comprueban que cada funcionalidad requerida se comporta como fue especificada, ejecutando un entrenamiento completo sobre el entorno simulado. Las dos primeras se escriben con el arnés de pruebas nativo de Rust y se ejecutan mediante `cargo test`; las de aceptación se levantan sobre los entornos contenedizados descritos más abajo, orquestados por los scripts de Docker del repositorio. Las tres son automatizadas y de ejecución desatendida, y se versionan junto con el código, de modo que cada cambio pueda validarse antes de integrarse al trabajo.

En cuanto a los roles, la especificación y el mantenimiento de las pruebas están a cargo de los tres integrantes del equipo: cada uno escribe las pruebas correspondientes a los componentes sobre los que concentra su foco, según se detalla en los Aportes individuales, y la revisión de esas pruebas queda a cargo del integrante que revisa el *pull request* asociado. No se contemplan pruebas manuales.

Lo aquí descrito constituye el plan de validación en esta instancia de la propuesta, y podrá ser ajustado a medida que avance el desarrollo del trabajo.

Métricas de comparación. Para comparar los algoritmos implementados se consideran tres métricas principales: el tiempo total de ejecución del entrenamiento, la convergencia de la optimización, seguida a través de la evolución de la función de costo y de la exactitud alcanzada, y la escalabilidad, entendida como la variación del rendimiento a medida que cambia la cantidad de nodos que participan. A estas se suma el comportamiento del sistema en configuraciones heterogéneas, en las que los nodos disponen de capacidades de cómputo distintas.

Entornos y reproducibilidad. Las simulaciones se ejecutan sobre entornos contenedizados con Docker, lo que permite reproducir de forma controlada distintas configuraciones de nodos y limitar los recursos de cómputo asignados a cada uno para emular máquinas heterogéneas. Los resultados obtenidos se recopilan, analizan y grafican con Python, y se documentan junto con la configuración que los generó para que las comparaciones sean reproducibles.

14 Plan de actividades

El proceso de construcción del producto sigue el método incremental descrito en la Metodología, del que este plan es la concreción: la gestión del alcance, de los tiempos, de los riesgos, de los indicadores de calidad y de las reuniones con los tutores se rige por lo allí establecido. Las tareas que siguen se abordan en iteraciones sucesivas, priorizando primero el sistema base y las implementaciones de referencia, y dejando las líneas de optimización para las iteraciones posteriores.

Las principales tareas para llevar a cabo el desarrollo de este trabajo son:

1. Leer y analizar los trabajos previos, actuales y que surjan sobre el entrenamiento distribuido de modelos de aprendizaje profundo.
2. Investigar sobre la implementación de TensorFlow distribuido.
3. Investigar sobre la implementación distribuida de Pytorch.

Tanto TensorFlow como PyTorch son implementaciones previas del tipo de sistema que se desarrollará en este trabajo.

4. Desarrollar el sistema distribuido que sirva como base para la implementación y análisis de los algoritmos actuales en Rust. Esto comprende el núcleo de redes neuronales (capas, optimizadores, funciones de pérdida y la representación de tensores), el manejo y la distribución de los conjuntos de datos, el middleware de comunicación entre nodos y los procesos de servidor y de trabajador. Este sistema proveerá una base sobre la cual poder probar distintos algoritmos de entrenamiento distribuido de modelos de machine learning. La idea es hacerlo tan *parametrizable* como sea posible para facilitar la posterior investigación y el desarrollo de estrategias que optimicen los tiempos de ejecución.
5. Implementar *Parameter Server* sobre el sistema desarrollado en 4.
6. Implementar *All-Reduce* sobre el sistema desarrollado en 4.
7. Implementar *Strategy-Switch* sobre el sistema desarrollado en 4. y utilizando las implementaciones de los algoritmos en 5. y 6.

Estos tres últimos puntos refieren al punto de partida de la implementación del sistema funcional y servirán como referencia para la comparación con las futuras mejoras que se estudien y desarrollen.

8. Estudiar sobre optimizaciones de comunicación entre nodos e implementarlas.
9. Estudiar sobre optimizaciones de sincronización de las copias del modelo en los distintos nodos e implementarlas.
10. Implementar una interfaz funcional externa para poder usar el sistema en Python, y una interfaz interactiva de terminal que permita configurar, lanzar y monitorear en vivo los entrenamientos. La idea de este punto es proveer una API fácil de usar para aquellos usuarios que trabajen con modelos de machine learning en este lenguaje, siendo que Python es el lenguaje más popular para este tipo de proyectos.
11. Testear el sistema desarrollado, con tests unitarios y de integración.
12. Simular la ejecución de los algoritmos implementados sobre el sistema distribuido base en distintas configuraciones de nodos; esto es, lograr una métrica que muestre el rendimiento del sistema, según los parámetros que este use, para distintas combinaciones de máquinas, con distintas capacidades de cómputo, que trabajen en la ejecución.
13. Estudiar sobre optimizaciones de carga de cómputo en configuraciones heterogéneas e implementarlas.

14. Analizar los resultados obtenidos de la comparación de los algoritmos, documentar y volcar el análisis utilizando gráficos en Python. Esto abarca también los resultados obtenidos en las simulaciones mencionadas en 12.
15. Documentar el código generado y el proceso de desarrollo (decisiones que se tomaron, inconvenientes encontrados, etc.).
16. Realizar un informe detallado de la evolución del trabajo y los resultados obtenidos.

14.1 Entregables e hitos de avance

El trabajo se organiza en torno a cuatro hitos, cada uno asociado a un entregable verificable:

1. **Relevamiento del estado del arte** (tareas 1 a 3). Entregable: un documento de análisis de los trabajos previos y de las implementaciones distribuidas de TensorFlow y PyTorch, que fundamenta las decisiones de diseño del sistema.
2. **Sistema base con implementaciones de referencia** (tareas 4 a 7, 11). Entregable: el sistema distribuido en Rust, con *Parameter Server*, *All-Reduce* y *Strategy-Switch* funcionando sobre él, acompañado de su conjunto de pruebas unitarias y de integración. Este hito coincide con la entrega de medio término del Trabajo Profesional.
3. **Optimizaciones e interfaces** (tareas 8 a 10, 13). Entregable: las optimizaciones de comunicación, de sincronización y de carga en configuraciones heterogéneas, junto con la biblioteca de Python construida sobre la FFI y la interfaz interactiva de terminal.
4. **Experimentación e informe final** (tareas 12, 14 a 16). Entregable: las simulaciones sobre las distintas configuraciones de nodos, el análisis comparativo de los algoritmos con sus gráficos, la documentación del código y del proceso de desarrollo, y el informe final del trabajo.

El avance de cada hito se revisa en las reuniones semanales con los tutores, y el pasaje de un hito al siguiente requiere que los entregables del anterior hayan superado los criterios de aceptación definidos en la Metodología.

14.2 Carga horaria

La columna de responsables indica el o los integrantes que concentran el foco principal de cada tarea, de acuerdo con lo detallado en los Aportes individuales; las etapas de análisis, revisión de código y pruebas involucran a los tres integrantes en todas las tareas. Se identifica con **A** a Alejo Ordoñez, con **L** a Lorenzo Minervino y con **M** a Marcos Bianchi Fernández. Las horas indicadas corresponden al esfuerzo total del equipo en cada tarea.

Nro.	Tarea	Duración (horas)	Responsables
1	Leer y analizar los trabajos previos, actuales y que surjan	100	A, L y M
2	Investigar sobre la implementación de <i>TensorFlow</i> distribuido	50	A, L y M
3	Investigar sobre la implementación distribuida de <i>Pytorch</i>	50	A, L y M
4	Desarrollar el sistema distribuido en Rust	150	A, L y M
5	Implementar <i>Parameter Server</i>	100	M
6	Implementar <i>All-Reduce</i>	100	L
7	Implementar <i>Strategy-Switch</i>	100	L y M
8	Estudiar sobre optimizaciones de comunicación entre nodos e implementarlas	150	M
9	Estudiar sobre optimizaciones de sincronización de las copias del modelo en los distintos nodos e implementarlas	150	L y M
10	Implementar la interfaz de funciones externas en Python y la interfaz interactiva de terminal	50	L
11	Testear el sistema desarrollado, con tests unitarios y de integración	50	A, L y M
12	Simular la ejecución de los algoritmos en distintas configuraciones de nodos	100	L
13	Estudiar sobre optimizaciones de carga de cómputo en configuraciones heterogéneas e implementarlas	150	A y M
14	Analizar los resultados obtenidos y volcar el análisis utilizando gráficos en Python	100	L
15	Documentar el código generado y el proceso de desarrollo	50	A, L y M
16	Escribir el informe final	50	A, L y M
	Total	1500	

15 Aportes individuales

El desarrollo del trabajo se plantea como una responsabilidad compartida: las etapas de análisis, implementación y pruebas son llevadas adelante por los tres integrantes, tal como se refleja en el Plan de actividades. Sin perjuicio de ello, cada integrante concentra su foco principal en un conjunto de tareas, que se detalla a continuación.

Alejo Ordoñez (Padrón 108397).

- Núcleo de machine learning: modelo, capas, optimizadores y funciones de pérdida.
- Capas convolucionales y max pooling.
- Manejo y distribución de datasets.

Lorenzo Minervino (Padrón 107863).

- Módulo del Worker y estrategia All-Reduce.
- Interfaz de funciones externas (FFI) en Python y la TUI.
- Benchmarks y análisis experimental de resultados.

Marcos Bianchi Fernández (Padrón 108921).

- Algoritmos de entrenamiento distribuido.
- Módulo de Comunicaciones.
- Módulo del Servidor.

16 Referencias

- Developers, T. (2021). *TensorFlow*. Zenodo. <https://doi.org/10.5281/zenodo.4758419>
- Li, M., Andersen, D. G., Park, J. W., Smola, A. J., Ahmed, A., Josifovski, V., ... Su, B.-Y. (2014). Scaling distributed machine learning with the parameter server. *Proceedings of the 11th USENIX Conference on Operating Systems Design and Implementation*, 583-598. USA: USENIX Association.
- Li, Z., Davis, J., & Jarvis, S. (2017). An Efficient Task-based All-Reduce for Machine Learning Applications. *Proceedings of the Machine Learning on HPC Environments*. New York, NY, USA: Association for Computing Machinery. <https://doi.org/10.1145/3146347.3146350>
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... Chintala, S. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Recuperado de <https://arxiv.org/abs/1912.01703>
- Provatas, N., Chalas, I., Konstantinou, I., & Koziris, N. (2025). Strategy-Switch: From All-Reduce to Parameter Server for Faster Efficient Training. *IEEE Access, PP*, 1-1. <https://doi.org/10.1109/ACCESS.2025.3528248>

17 Anexos

17.1 Anexo A: Repositorio del proyecto

El código fuente del trabajo, junto con su documentación técnica y el historial completo de su desarrollo, se encuentra alojado en el siguiente repositorio público:

`https://github.com/lminervino18/oxidized-neural-orchestra`

El repositorio está organizado como un *workspace* de Cargo, la herramienta de construcción de Rust, y se divide en los siguientes módulos:

- **machine_learning**: el núcleo de redes neuronales, que comprende las capas, los optimizadores, las funciones de pérdida y la representación de tensores.
- **comms**: el middleware de comunicación entre nodos.
- **parameter_server**: la implementación de la estrategia *Parameter Server*, en sus variantes sincrónica y asincrónica.
- **worker**: el runtime del nodo trabajador, que contiene además la implementación en anillo de *All-Reduce*.
- **orchestrator**: el plano de control, que asigna los roles de servidor y de trabajador y dirige la ejecución de un entrenamiento.
- **node**: el proceso de cómputo, agnóstico del rol que le sea asignado.
- **orchestra-py**: la biblioteca de Python, construida sobre la interfaz de funciones externas.
- **orchestui**: la interfaz interactiva de terminal.
- **docker**: los scripts que levantan los entornos de simulación multinodo.
- **docs**: la presente propuesta y la documentación de diseño del sistema.